Test Case 1

1. University Scholarship Allocation

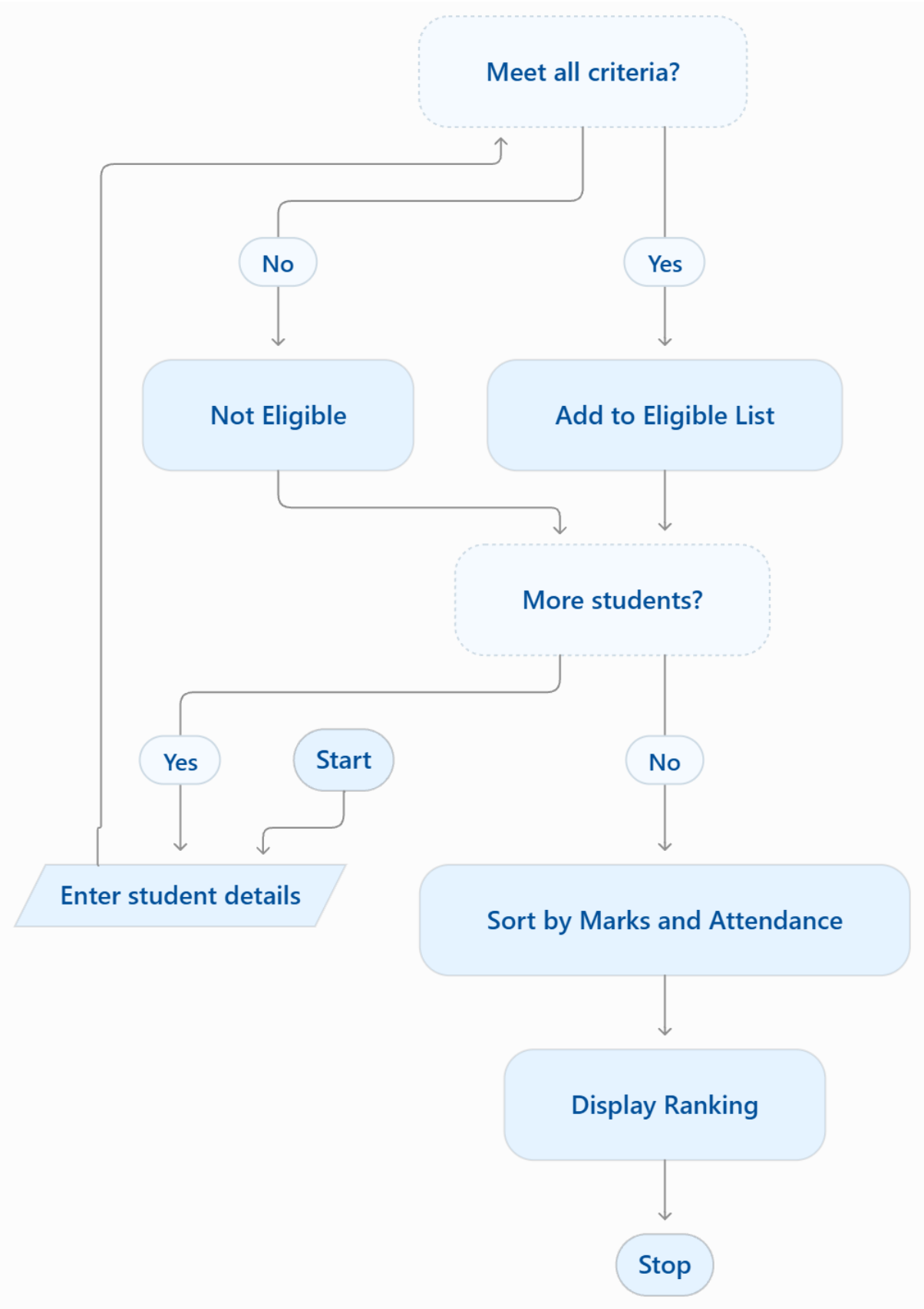
1.Problem Statement

To develop a Python program that checks students' marks, subject scores, attendance, and family income to determine scholarship eligibility and rank eligible students.

2.Algorithm

1. Start the program.
2. Read student details.
3. Check marks, subject score, attendance, and income.
4. Select eligible students.
5. Rank students by aggregate marks; break ties using attendance.
6. Display the ranking and stop.

3.Flowchart



4.source code

```
n = int(input("Number of students: "))
students = []

for i in range(n):
    name = input("Name: ")
    marks = float(input("Aggregate marks: "))
    subject = float(input("Minimum subject score: "))
    attendance = float(input("Attendance %: "))
    income = float(input("Family income: "))

    if marks >= 75 and subject >= 40 and attendance >= 75 and income <= 200000:
        students.append((name, marks, attendance))

students.sort(key=lambda x: (-x[1], -x[2]))

print("\nScholarship Ranking:")
for rank, s in enumerate(students, 1):
    print(rank, s[0], "Marks:", s[1], "Attendance:", s[2])

if not students:
    print("No eligible students.")
```

5.Sample Input

```
Number of students: 2
Name: Ravi
Aggregate marks: 85
Minimum subject score: 60
Attendance %: 90
Family income: 150000
Name: Sita
Aggregate marks: 85
Minimum subject score: 55
Attendance %: 95
Family income: 180000
```

6.Sample Output

```
Scholarship Ranking:
1 Sita Marks: 85.0 Attendance: 95.0
2 Ravi Marks: 85.0 Attendance: 90.0
```

Test Case 2

11. Examination Result and Grace-Mark Validation

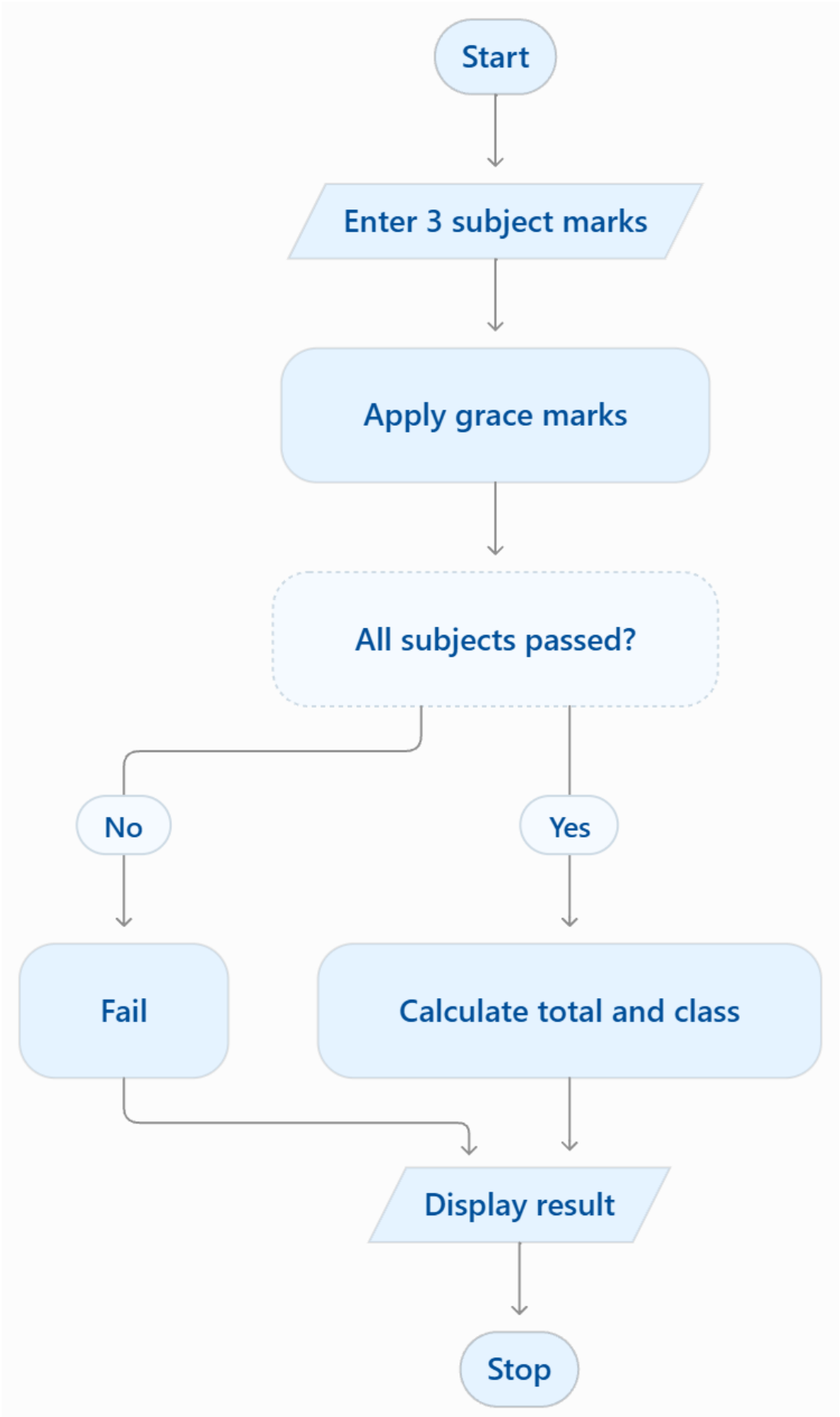
1. Problem Statement

To process subject marks, apply grace marks if needed, check pass criteria, and display the final result and classification without exceeding maximum marks.

2. Algorithm

1. Read marks of three subjects.
2. Apply grace marks if required.
3. Check pass marks for each subject.
4. Display total, result, and classification.

3. Flowchart


Start
Enter 3 subject marks
Apply grace marks
All subjects passed?
No
Yes
Fail
Calculate total and class
Display result
Stop

4.source code

```
marks = list(map(int, input("Enter 3 marks: ").split()))


for i in range(3):
    if marks[i] == 34:
        marks[i] = min(35, marks[i] + 1)


total = sum(marks)


if min(marks) < 35:
    result = "Fail"
elif total >= 240:
    result = "Distinction"
elif total >= 180:
    result = "First Class"
else:
    result = "Pass"


print("Marks:", marks)
print("Total:", total, "/ 300")
print("Result:", result)
```

5.Sample Input

```
Plain text

Enter 3 marks: 34 80 90
```

6. Sample Output

```
Plain text

Marks: [35, 80, 90]
Total: 205 / 300
Result: First Class
```

Test Case 3

21. Traffic Violation Penalty Calculator

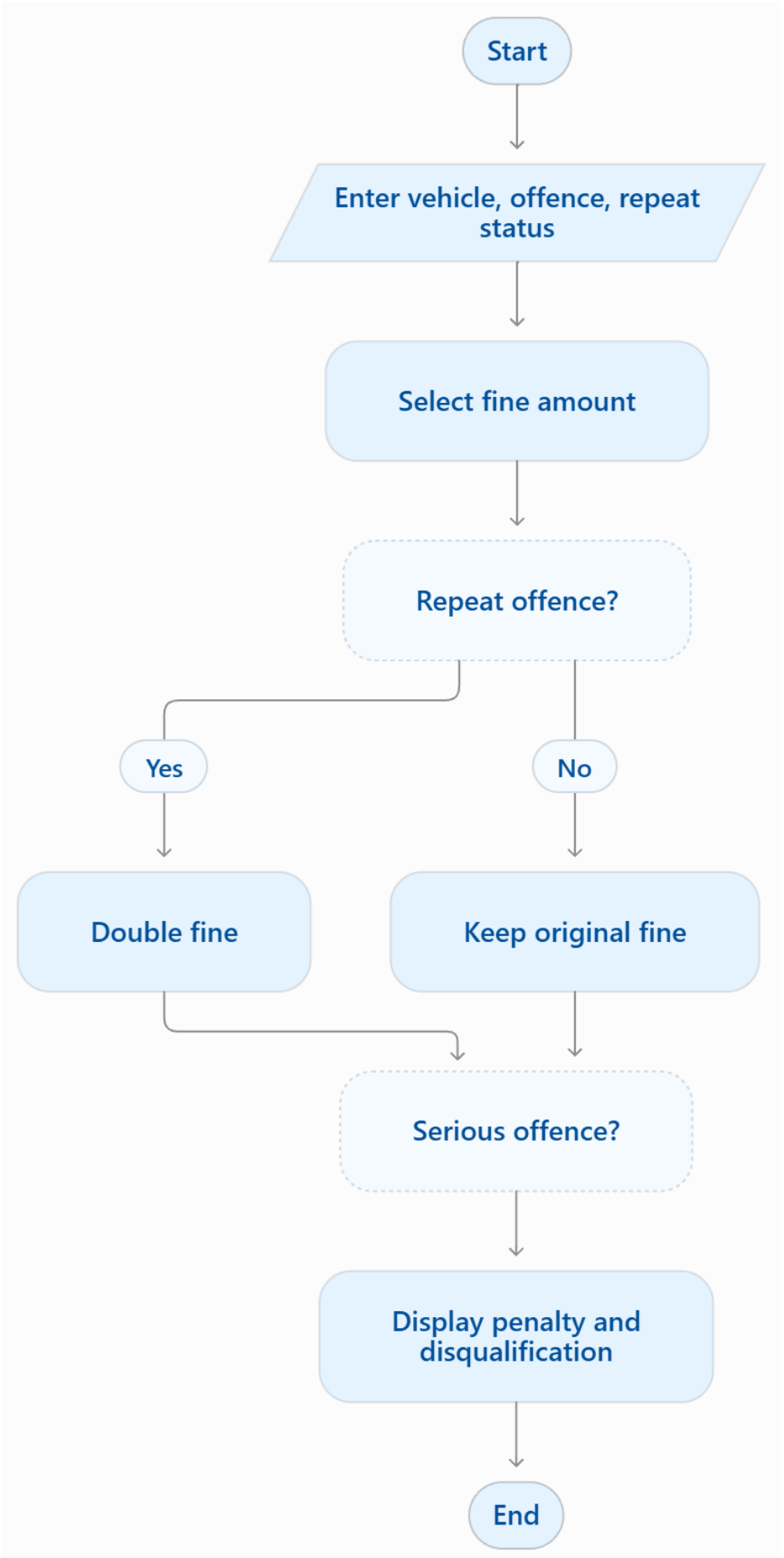
1. Problem Statement

Calculate traffic fines based on vehicle category, offence type, and repeat offences. Identify whether the driver is disqualified.

2. Algorithm

1. Read vehicle category, offence, and repeat status.
2. Select the fine amount.
3. Double the fine for repeat offences.
4. Display the total penalty and disqualification status.

3. Flowchart


Start
Enter vehicle, offence, repeat status
Select fine amount
Repeat offence?
Yes
No
Double fine
Keep original fine
Serious offence?
Display penalty and disqualification
End

4.source code

```
fine = {"Car": 1000, "Bike": 500, "Truck": 2000}


v = input("Vehicle (Car/Bike/Truck): ")
o = input("Offence (Normal/Serious): ")
r = input("Repeat (yes/no): ")


penalty = fine.get(v, 0)
if r == "yes":
    penalty *= 2


print("Penalty: ₹", penalty)
print("Disqualification:", "Yes" if o == "Serious" else "No")
```

5.Sample Input

```
Plain text

Vehicle (Car/Bike/Truck): Car
Offence (Normal/Serious): Serious
Repeat (yes/no): yes
```

6. Sample Output

```
Plain text

Penalty: ₹ 2000
Disqualification: Yes
```

Test Case 4

31. String-Based Examination Answer Evaluation

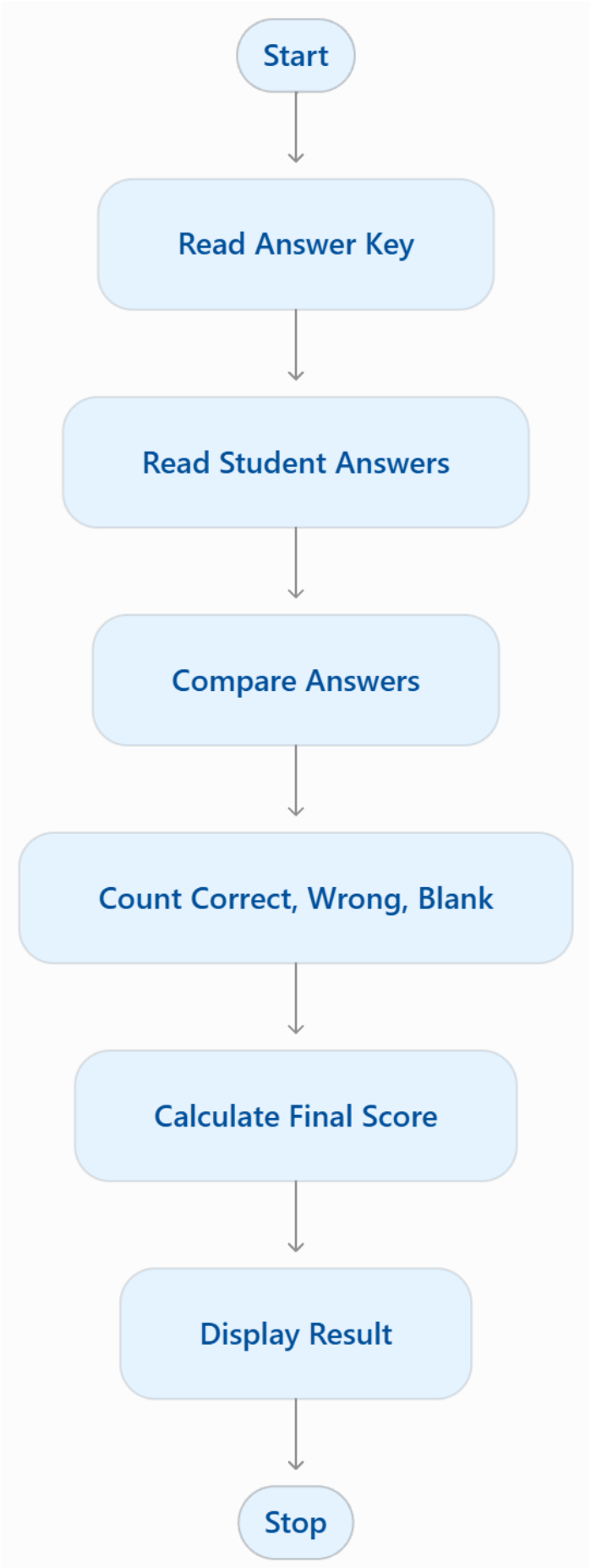
1. Problem Statement

Compare student answers with the answer key, calculate correct, incorrect, and unanswered responses, apply negative marking, and display the final result.

2. Algorithm

1. Start.
2. Read the answer key and student answers.
3. Count correct, incorrect, and unanswered answers.
4. Apply negative marking for incorrect answers.
5. Display the final score.
6. Stop.

3. Flowchart


Start
Read Answer Key
Read Student Answers
Compare Answers
Count Correct, Wrong, Blank
Calculate Final Score
Display Result
Stop

4.source code

```
key = input("Answer key: ").split()
ans = input("Student answers: ").split()


c = w = u = 0
for k, a in zip(key, ans):
    if a == "-":
        u += 1
    elif a == k:
        c += 1
    else:
        w += 1


score = c - w * 0.25
print("Correct:", c, "Wrong:", w, "Unanswered:", u)
print("Final Score:", score)
```

5.Sample Input

```
Plain text

Answer key: A B C D
Student answers: A X - D
```

6. Sample Output

```
Plain text

Correct: 2 Wrong: 1 Unanswered: 1
Final Score: 1.75
```

Test Case 5

41. Nested List Classroom Occupancy Analysis

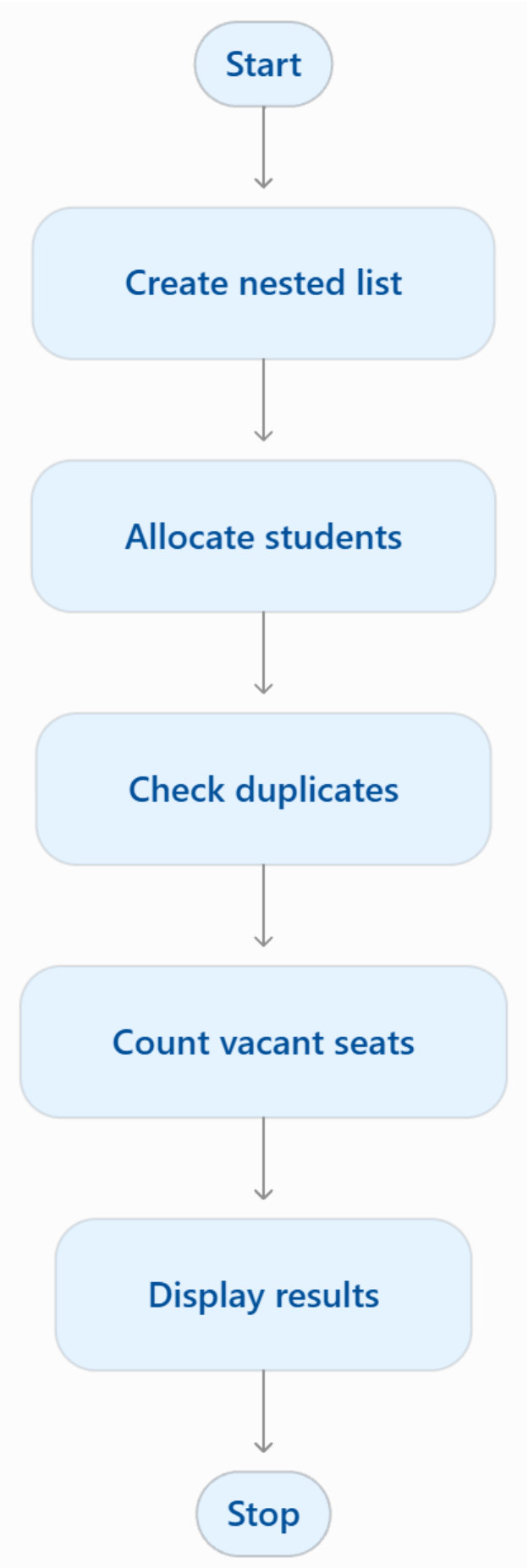
1. Problem Statement

Represent classrooms using nested lists, allocate students, preserve reserved seats, detect duplicate assignments, and calculate vacant seats.

2. Algorithm

1. Create a nested list of classroom seats.
2. Allocate students to available seats.
3. Preserve reserved seats.
4. Check duplicate student names.
5. Display vacant seat count.

3. Flowchart


Start
Create nested list
Allocate students
Check duplicates
Count vacant seats
Display results
Stop

4.source code

```
rooms = [["Ram", "Reserved", None],
         ["Sita", "Ram", None]]


students = [s for room in rooms for s in room
            if s not in (None, "Reserved")]


print("Duplicate:", len(students) != len(set(students)))
print("Vacant seats:", sum(s is None for r in rooms for s in r))
```

5.Sample Input

```
Plain text

Classroom 1: Ram, Reserved, Empty
Classroom 2: Sita, Ram, Empty
```

6. Sample Output

```
Plain text

Duplicate: True
Vacant seats: 2
```

Test Case 6

51. Bus Passenger Occupancy Simulation

1. Problem Statement

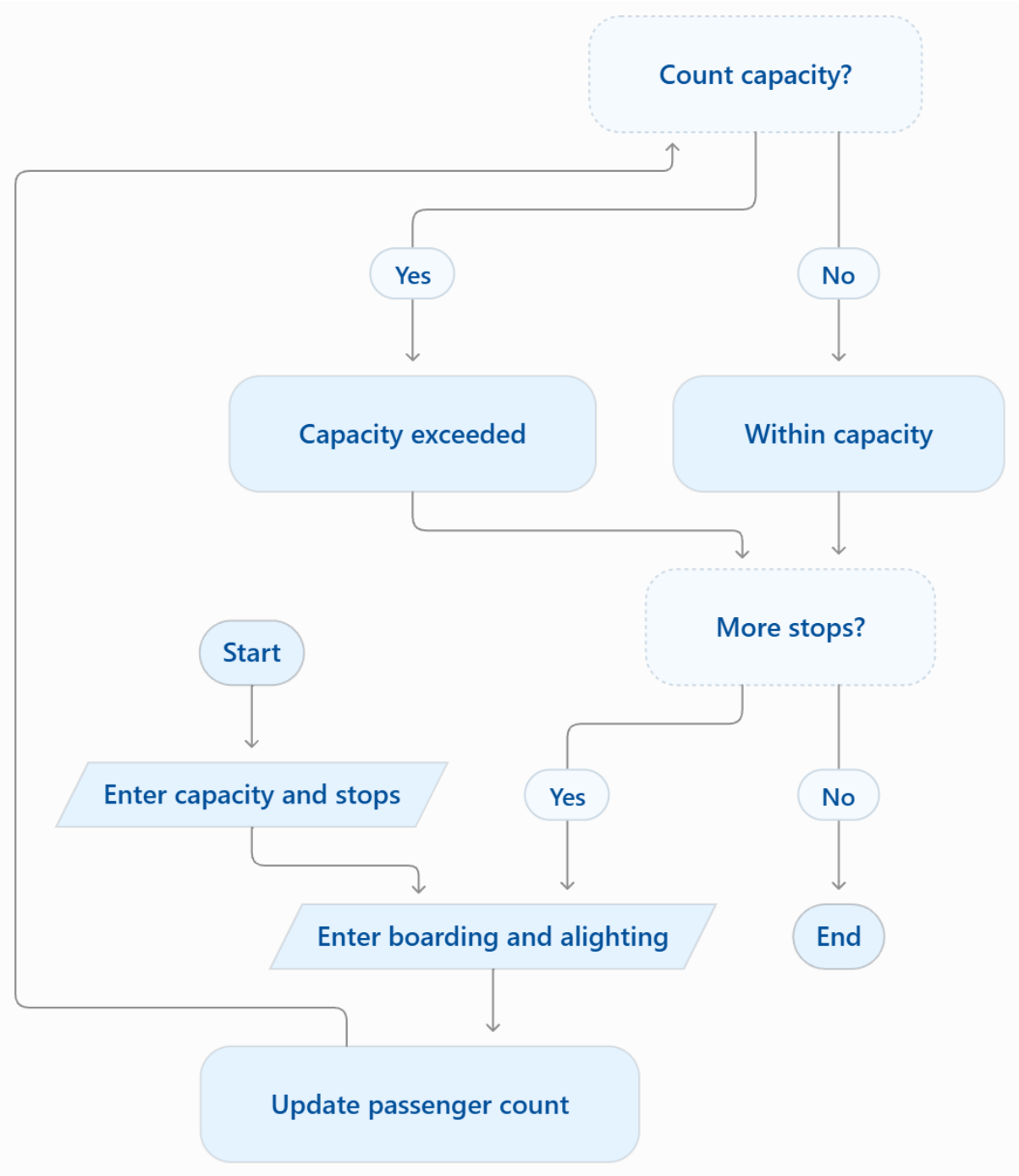
To calculate passenger counts at each bus stop using boarding and alighting numbers and check whether bus capacity is exceeded.

2. Algorithm

1. Read bus capacity and number of stops.

2. Read boarding and alighting counts.
3. Calculate current passengers.
4. Display passenger count and capacity status.

3. Flowchart



4.source code

```
cap = int(input("Bus capacity: "))
```

```
n = int(input("Number of stops: "))
p = 0

for i in range(n):
    b = int(input("Boarding: "))
    a = int(input("Alighting: "))
    p = max(0, p - a) + b
    print("Passengers:", p)
    print("Capacity exceeded" if p > cap else "Within capacity")
```

5.Sample Input

```
Plain text

Bus capacity: 40
Number of stops: 2
Boarding: 20
Alighting: 0
Boarding: 25
Alighting: 5
```

6. Sample Output

```
Plain text

Passengers: 20
Within capacity
Passengers: 40
Within capacity
```

Test Case 7

61. Nested List Examination Seating Validation

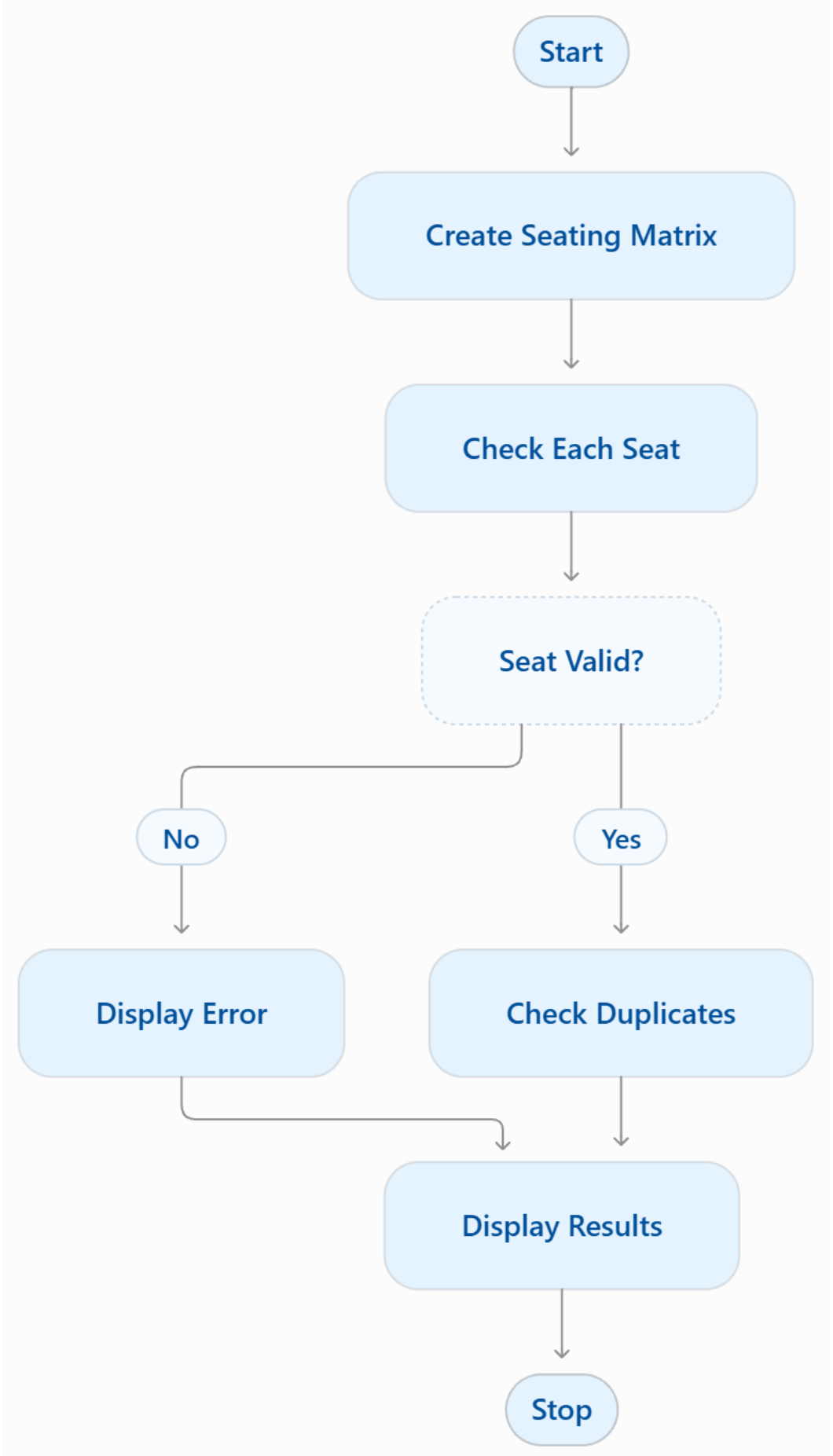
1. Problem Statement

Validate the seating matrix for duplicate roll numbers, reserved seats, empty seats, and invalid student allocations.

2. Algorithm

1. Store seats in a nested list.
2. Check each seat for reserved, empty, or invalid values.
3. Find duplicate roll numbers.
4. Display the validation results.

3. Flowchart


Start
Create Seating Matrix
Check Each Seat
Seat Valid?
No
Yes
Display Error
Check Duplicates
Display Results
Stop

4.source code

```
seats = [[101, 102], [103, 101]]
reserved = [(0, 1)]
seen = []

for i, row in enumerate(seats):
    for j, s in enumerate(row):
        if (i, j) in reserved:
            print(s, "Reserved seat occupied")
        elif s == 0:
            print("Empty seat")
        elif s < 0:
            print("Invalid allocation")
        elif s in seen:
            print(s, "Duplicate roll number")
        else:
            seen.append(s)
            print(s, "Valid")
```

5.Sample Input

```
Plain text

Seating Matrix: [[101, 102], [103, 101]]
Reserved seat: (0, 1)
```

6. Sample Output

```
Plain text

101 Valid
102 Reserved seat occupied
103 Valid
101 Duplicate roll number
```

Test Case 8

71. Integrated University Fee and Eligibility Audit

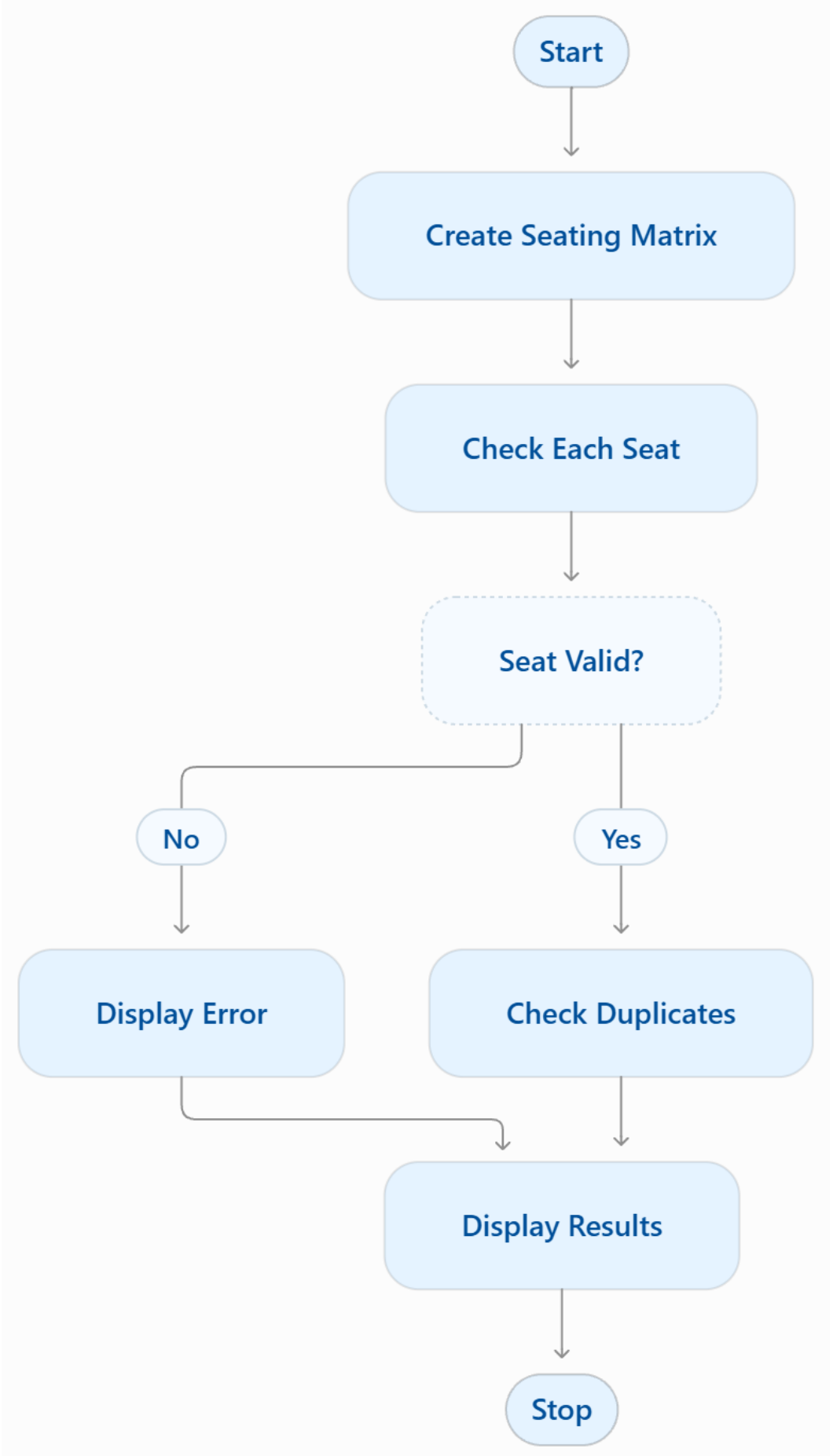
1. Problem Statement

To check students' fee payments, attendance, and internal marks to determine examination eligibility and report pending conditions.

2. Algorithm

1. Start.
2. Enter student details.
3. Check fees, attendance, and marks.
4. Display pending conditions or eligibility.
5. Stop.

3. Flowchart


Start
Create Seating Matrix
Check Each Seat
Seat Valid?
No
Yes
Display Error
Check Duplicates
Display Results
Stop

4.source code

```
seats = [[101, 102], [103, 101]]
reserved = [(0, 1)]
seen = []

for i, row in enumerate(seats):
    for j, s in enumerate(row):
        if (i, j) in reserved:
            print(s, "Reserved seat occupied")
        elif s == 0:
            print("Empty seat")
        elif s < 0:
            print("Invalid allocation")
        elif s in seen:
            print(s, "Duplicate roll number")
        else:
            seen.append(s)
            print(s, "Valid")
```

5.Sample Input

```
Plain text

Seating Matrix: [[101, 102], [103, 101]]
Reserved seat: (0, 1)
```

6. Sample Output

```
Plain text

101 Valid
102 Reserved seat occupied
103 Valid
101 Duplicate roll number
```

Test Case 9

81. Campus Event Seating and Entry Validation

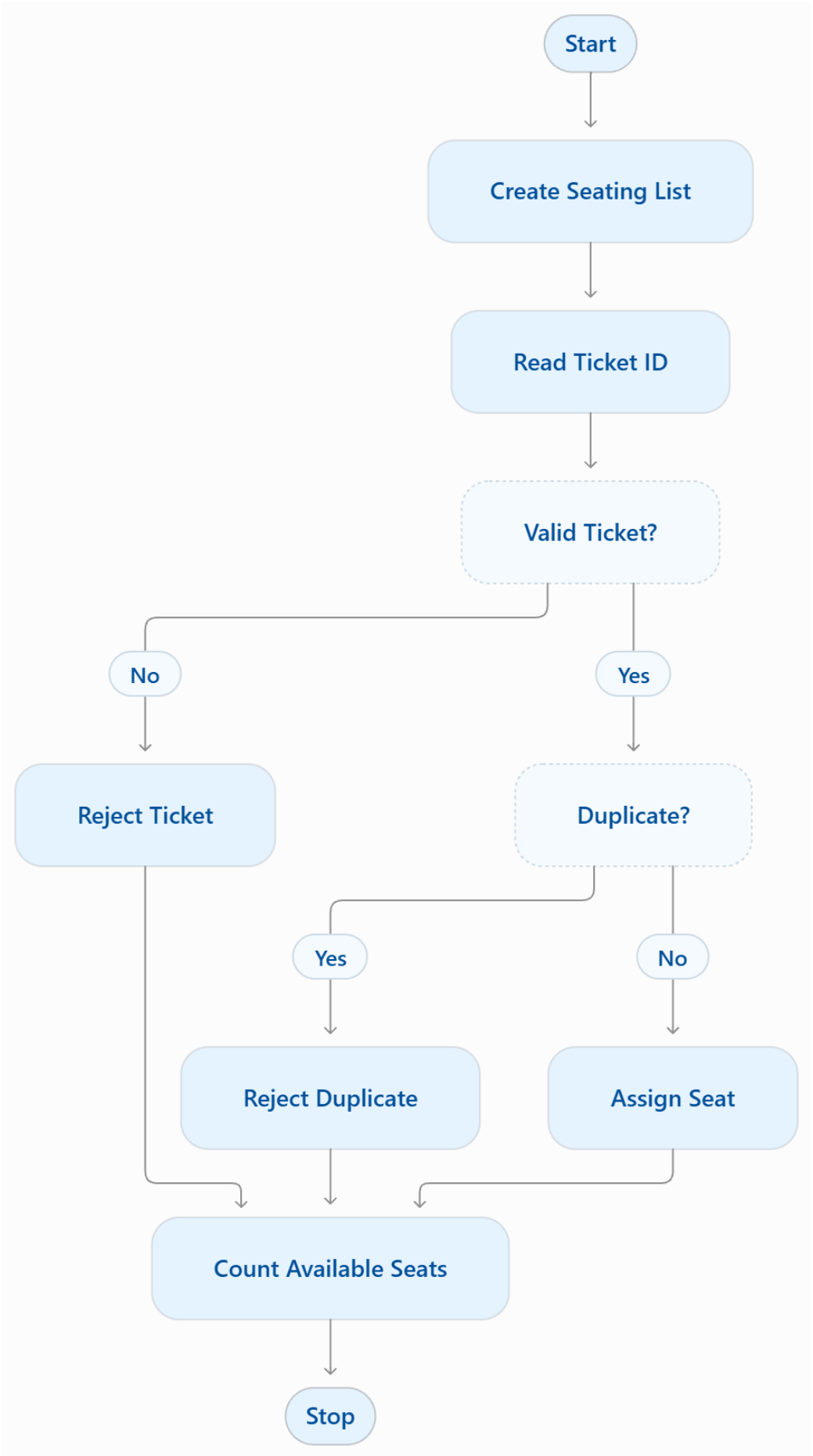
1. Problem Statement

To assign seats using nested lists, validate tickets, detect duplicate entries, and calculate available seats.

2. Algorithm

1. Create a seating list and registration IDs.
2. Read ticket IDs.
3. Check valid and duplicate tickets.
4. Assign seats and count available seats.
5. Display the results.

3. Flowchart


Start
Create Seating List
Read Ticket ID
Valid Ticket?
No
Yes
Reject Ticket
Duplicate?
Yes
No
Reject Duplicate
Assign Seat
Count Available Seats
Stop

4.source code

```
seats = [[None, None], [None, None]]
registered = ["A1", "A2", "A3"]
entered = []

for ticket in ["A1", "A2", "A1", "X1"]:
    if ticket not in registered:
        print(ticket, "Invalid")
    elif ticket in entered:
        print(ticket, "Duplicate")
    else:
        entered.append(ticket)
        for row in seats:
            if None in row:
                row[row.index(None)] = ticket
                print(ticket, "Seat assigned")
                break

print("Available seats:", sum(r.count(None) for r in seats))
```

5.Sample Input

```
Plain text

A1, A2, A1, X1
```

6. Sample Output

```
Plain text

A1 Seat assigned
A2 Seat assigned
A1 Duplicate
X1 Invalid
Available seats: 2
```

Test Case 10

91. Advanced String-Based Registration Record Analyzer

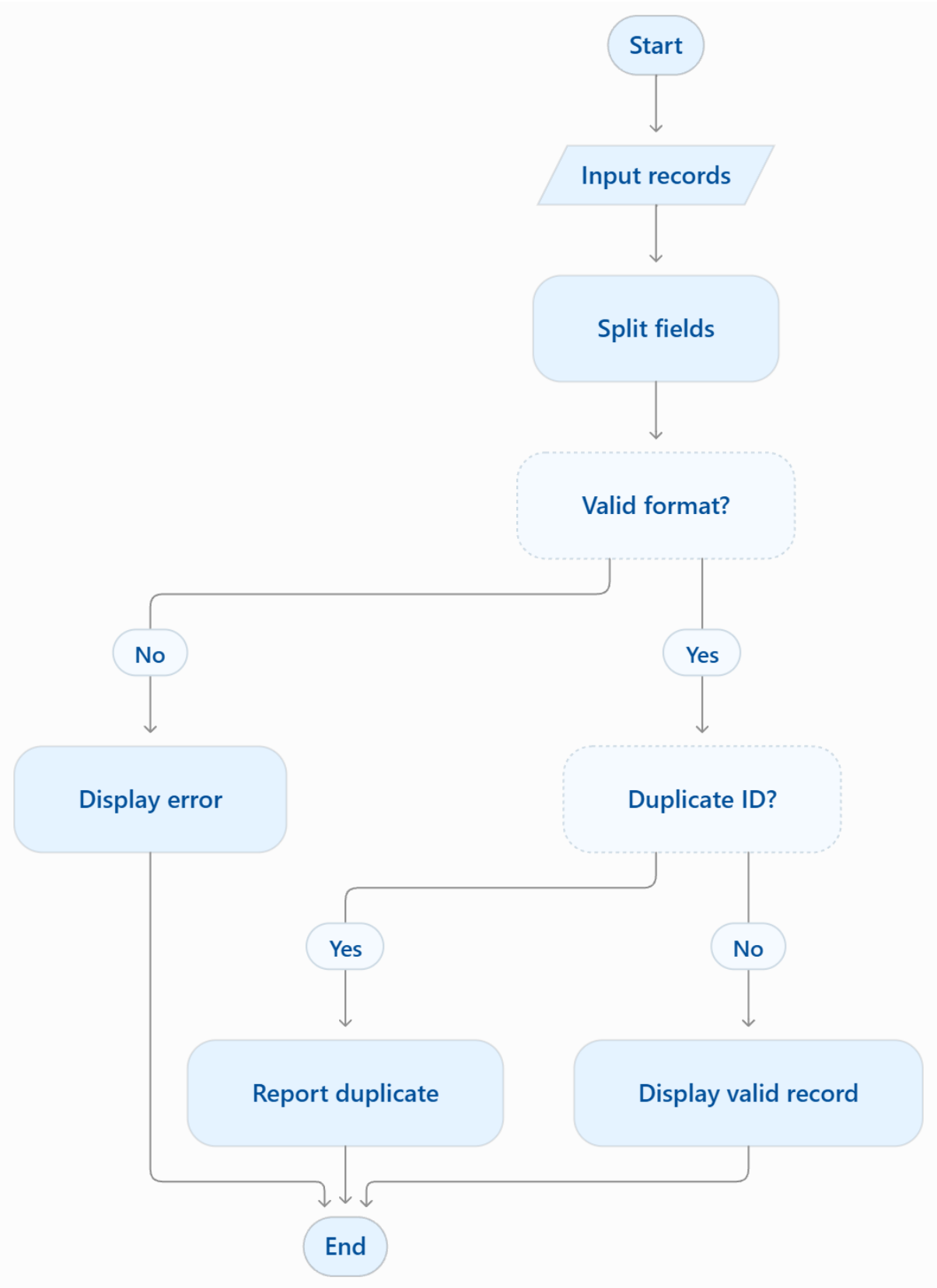
1. Problem Statement

To validate registration strings, check field formats and lengths, extract details, detect duplicates, and display errors.

2. Algorithm

1. Read registration records.
2. Split each record into fields.
3. Validate field lengths and formats.
4. Check duplicate IDs.
5. Display valid records and errors.

3. Flowchart



4.source code

```
records = input("Enter records: ").split(";")
seen = set()
```

```
for r in records:
    f = r.split(",")
    if len(f) != 3 or len(f[0]) != 4 or not f[1].isalpha():
        print(r, "- Invalid format")
    elif f[0] in seen:
        print(r, "- Duplicate ID")
    else:
        seen.add(f[0])
        print("Valid:", f[0], f[1], f[2])
```

5.Sample Input

```
Plain text

A101,Ravi,CSE;A102,Sita,ECE;A101,Ram,EEE;12,John,CSE
```

6. Sample Output

```
Plain text

Valid: A101 Ravi CSE
Valid: A102 Sita ECE
A101,Ram,EEE - Duplicate ID
12,John,CSE - Invalid format
```